

Transformer Architectures and Variants

A Mathematical and Architectural Treatment

Part 4

Vision, Image, and Multimodal Transformer Architectures

Contents

1	Transformers Beyond Text	1
2	Vision Transformer (ViT)	2
2.1	Why ViT Was Introduced	2
2.2	Image as Input	2
2.3	Patch Extraction	2
2.4	Linear Patch Projection	3
2.5	Class Token	3
2.6	Position Embeddings	3
2.7	Transformer Encoder	4
2.8	Classification	4
2.9	Complete ViT Pipeline	5
2.10	What Changed Relative to the Original Transformer	5
2.11	Applications	5
3	DeiT	6
3.1	Motivation	6
3.2	Teacher and Student	6
3.3	Distillation Loss	6
3.4	Distillation Token	7
3.5	Outputs	7
4	Swin Transformer	8
4.1	Why Swin Was Introduced	8
4.2	Patch Representation	8
4.3	Window Partition	8
4.4	Complexity	9
4.5	The Shifted Window Mechanism	9
4.6	Mathematical Progression	10
4.7	Hierarchical Representation	10
4.8	Why the Hierarchy Matters	10
4.9	Applications	11
5	Pyramid Vision Transformer	12
5.1	Motivation	12

5.2	Hierarchical Input	12
5.3	Spatial-Reduction Attention	12
5.4	Applications	13
6	Deformable Attention Transformer	14
6.1	Motivation	14
6.2	Reference Position	14
6.3	Sparse Sampling	14
7	DETR	16
7.1	Motivation	16
7.2	Image Encoder	16
7.3	Object Queries	16
7.4	Decoder Cross-Attention	17
7.5	Prediction Heads	17
7.6	Set Prediction	17
7.7	Architectural Significance	18
8	MaskFormer	19
8.1	Motivation	19
8.2	Pixel Features	19
8.3	Mask Queries	19
8.4	Mask Prediction	20
9	Multimodal Transformers	21
9.1	Cross-Attention	21
10	CLIP	23
10.1	Motivation	23
10.2	Image Encoder	23
10.3	Text Encoder	23
10.4	Similarity	24
10.5	Contrastive Learning	24
10.6	Zero-Shot Classification	24
11	BLIP and Vision-Language Transformer Architectures	26
11.1	Motivation	26
11.2	Vision Encoder	26
11.3	Text Representation	26
11.4	Image-Text Interaction	27
12	BEiT	28
12.1	Motivation	28
12.2	Image Tokenization	28
12.3	Masking	28
12.4	Prediction	29

13 MAE: Masked Autoencoders	30
13.1 Motivation	30
13.2 Patch Sequence	30
13.3 Encoder	30
13.4 Decoder	31
13.5 Pixel Reconstruction	31
13.6 Why the Asymmetry Matters	31
14 DINO and Self-Supervised Vision Transformers	32
14.1 Motivation	32
14.2 Multiple Views	32
14.3 Student and Teacher	32
14.4 Architectural Significance	33
15 Overall Architectural Progression	34
15.1 ViT	34
15.2 Swin	34
15.3 PVT	34
15.4 DETR	34
15.5 CLIP	35
15.6 MAE	35
15.7 Multimodal Transformers	35

Chapter 1

Transformers Beyond Text

The original Transformer was introduced for sequence-to-sequence language translation.

Let a token sequence be

$$X = (x_1, x_2, \dots, x_n). \quad (1.1)$$

After embedding,

$$H^{(0)} \in \mathbb{R}^{n \times d}. \quad (1.2)$$

Self-attention then produces interactions between positions:

$$Q = H^{(0)} W^Q, \quad (1.3)$$

$$K = H^{(0)} W^K, \quad (1.4)$$

$$V = H^{(0)} W^V. \quad (1.5)$$

The attention output is

$$Z = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V. \quad (1.6)$$

Images do not naturally arrive as one-dimensional token sequences. An image is instead represented as a two-dimensional array

$$I \in \mathbb{R}^{H \times W \times C}, \quad (1.7)$$

where H is height, W is width, and C is the number of channels.

Vision Transformers therefore require a transformation

$$\boxed{\text{image} \rightarrow \text{tokens} \rightarrow \text{Transformer}}. \quad (1.8)$$

The central question is how the image should be converted into tokens and how spatial information should be preserved.

Chapter 2

Vision Transformer (ViT)

2.1 Why ViT Was Introduced

The Vision Transformer treats an image as a sequence of image patches.

Paper:

An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale

The architecture removes the requirement that a convolutional neural network must be the primary image feature extractor.

2.2 Image as Input

Let the image be

$$I \in \mathbb{R}^{H \times W \times C}. \quad (2.1)$$

Choose a patch size

$$P \times P. \quad (2.2)$$

Assuming P divides both H and W , the number of patches is

$$N = \frac{HW}{P^2}. \quad (2.3)$$

For example, if

$$H = W = 224, \quad P = 16, \quad (2.4)$$

then

$$N = \frac{224 \times 224}{16^2} = 196. \quad (2.5)$$

2.3 Patch Extraction

Each patch is

$$x_p^{(i)} \in \mathbb{R}^{P \times P \times C}. \quad (2.6)$$

Flatten the patch:

$$\text{vec}\left(x_p^{(i)}\right) \in \mathbb{R}^{P^2 C}. \quad (2.7)$$

Thus the image becomes

$$X_p \in \mathbb{R}^{N \times P^2 C}. \quad (2.8)$$

2.4 Linear Patch Projection

Each flattened patch is projected into the Transformer embedding dimension:

$$z_i = x_i E, \quad (2.9)$$

where

$$E \in \mathbb{R}^{P^2 C \times D}. \quad (2.10)$$

Therefore,

$$Z \in \mathbb{R}^{N \times D}. \quad (2.11)$$

The operation is mathematically equivalent to learning a linear representation of every image patch.

2.5 Class Token

A learned class token

$$x_{\text{cls}} \in \mathbb{R}^D \quad (2.12)$$

is prepended:

$$Z' = [x_{\text{cls}}; z_1; z_2; \dots; z_N]. \quad (2.13)$$

Hence,

$$Z' \in \mathbb{R}^{(N+1) \times D}. \quad (2.14)$$

2.6 Position Embeddings

The Transformer itself does not inherently know the spatial location of a patch.

A positional embedding matrix is therefore added:

$$E_{\text{pos}} \in \mathbb{R}^{(N+1) \times D}. \quad (2.15)$$

The input to the Transformer becomes

$$H^{(0)} = Z' + E_{\text{pos}}. \quad (2.16)$$

2.7 Transformer Encoder

The sequence is processed by standard Transformer encoder blocks.

For layer l ,

$$Q^{(l)} = H^{(l)} W_l^Q, \quad (2.17)$$

$$K^{(l)} = H^{(l)} W_l^K, \quad (2.18)$$

$$V^{(l)} = H^{(l)} W_l^V. \quad (2.19)$$

Self-attention is

$$\text{SA}(H^{(l)}) = \text{softmax} \left(\frac{Q^{(l)} (K^{(l)})^T}{\sqrt{d_k}} \right) V^{(l)}. \quad (2.20)$$

The result passes through the Transformer block:

$$H^{(l+1)} = \text{TransformerBlock}(H^{(l)}). \quad (2.21)$$

2.8 Classification

After the final Transformer layer,

$$H^{(L)} \in \mathbb{R}^{(N+1) \times D}. \quad (2.22)$$

The class-token representation is

$$h_{\text{cls}} = H_0^{(L)}. \quad (2.23)$$

A classification head produces

$$y = W h_{\text{cls}} + b. \quad (2.24)$$

For classification probabilities,

$$p(y \mid I) = \text{softmax}(y). \quad (2.25)$$

2.9 Complete ViT Pipeline

The complete progression is

$$I \rightarrow \text{patch extraction} \rightarrow \text{flattening} \rightarrow \text{linear projection} \rightarrow \text{class token} \rightarrow \text{position embedding} \rightarrow \text{Transformer encoder} \quad (2.26)$$

2.10 What Changed Relative to the Original Transformer

The attention mechanism itself remains essentially the same.

The major change is the input representation:

$$\text{text tokens} \longrightarrow \text{image patches}. \quad (2.27)$$

Thus ViT demonstrates that the Transformer does not require language-specific tokens.

2.11 Applications

ViT is used for:

- image classification,
- image representation learning,
- object recognition,
- transfer learning,
- visual foundation models.

Repository:

[Vision Transformer implementation](#)

Chapter 3

DeiT

3.1 Motivation

Vision Transformers originally demonstrated strong performance when trained with sufficiently large datasets.

A central problem was therefore:

$$\text{How can ViT be trained effectively with less data?} \quad (3.1)$$

Data-efficient Image Transformers, or DeiT, introduces a training strategy based on knowledge distillation.

Paper:

Training data-efficient image transformers & distillation through attention

3.2 Teacher and Student

Let the teacher network produce

$$p_T(y \mid x). \quad (3.2)$$

The student Transformer produces

$$p_S(y \mid x). \quad (3.3)$$

The student is trained using both the true labels and the teacher's predictions.

3.3 Distillation Loss

A conventional classification loss is

$$\mathcal{L}_{\text{CE}} = - \sum_c y_c \log p_S(c \mid x). \quad (3.4)$$

A distillation loss can compare teacher and student probability distributions:

$$\mathcal{L}_{\text{KD}} = - \sum_c p_T(c \mid x) \log p_S(c \mid x). \quad (3.5)$$

The total objective can be written conceptually as

$$\mathcal{L} = \lambda \mathcal{L}_{\text{CE}} + (1 - \lambda) \mathcal{L}_{\text{KD}}. \quad (3.6)$$

3.4 Distillation Token

DeiT introduces a learned distillation token:

$$x_{\text{distill}}. \quad (3.7)$$

The Transformer input becomes

$$H^{(0)} = [x_{\text{cls}}; x_{\text{distill}}; z_1; \dots; z_N] + E_{\text{pos}}. \quad (3.8)$$

The class token and distillation token can therefore learn different information.

3.5 Outputs

At the final layer,

$$h_{\text{cls}} = H_0^{(L)}, \quad (3.9)$$

and

$$h_{\text{distill}} = H_1^{(L)}. \quad (3.10)$$

Separate heads can produce

$$p_{\text{cls}} = f_{\text{cls}}(h_{\text{cls}}) \quad (3.11)$$

and

$$p_{\text{distill}} = f_{\text{distill}}(h_{\text{distill}}). \quad (3.12)$$

The distillation token is therefore an architectural mechanism through which teacher information can influence the student.

Repository:

[DeiT implementation](#)

Chapter 4

Swin Transformer

4.1 Why Swin Was Introduced

Global self-attention over image patches has quadratic complexity with respect to the number of patches.

For an image containing N patches,

$$\text{global attention cost} \sim O(N^2 D). \quad (4.1)$$

For high-resolution images, N becomes very large.

Swin Transformer introduces hierarchical representations and window-based self-attention.

Paper:

Swin Transformer: Hierarchical Vision Transformer using Shifted Windows

4.2 Patch Representation

After patch partitioning,

$$H \in \mathbb{R}^{H_p \times W_p \times D}. \quad (4.2)$$

Instead of allowing every patch to attend to every other patch, the image is partitioned into local windows.

4.3 Window Partition

Let the window size be

$$M \times M. \quad (4.3)$$

A window contains

$$M^2 \quad (4.4)$$

tokens.

Self-attention is performed independently inside each window.

For one window,

$$X_w \in \mathbb{R}^{M^2 \times D}. \quad (4.5)$$

Then

$$Q_w = X_w W^Q, \quad (4.6)$$

$$K_w = X_w W^K, \quad (4.7)$$

$$V_w = X_w W^V. \quad (4.8)$$

Window attention becomes

$$Z_w = \text{softmax} \left(\frac{Q_w K_w^T}{\sqrt{d_k}} \right) V_w. \quad (4.9)$$

4.4 Complexity

Suppose there are

$$\frac{HW}{M^2} \quad (4.10)$$

windows.

Each window performs attention over M^2 tokens.

The approximate attention complexity becomes

$$O \left(\frac{HW}{M^2} M^4 D \right) = O(HW M^2 D). \quad (4.11)$$

Since M is fixed, this grows approximately linearly with image area:

$$O(HW). \quad (4.12)$$

This is substantially different from global attention:

$$O((HW)^2). \quad (4.13)$$

4.5 The Shifted Window Mechanism

If windows remain fixed, tokens belonging to different windows cannot communicate directly.

Swin therefore shifts the window partition between consecutive Transformer blocks.

For example, the first block uses

$$\text{regular windows}, \quad (4.14)$$

while the next uses

shifted windows. (4.15)

The shift allows information to cross the previous window boundaries.

4.6 Mathematical Progression

The sequence is approximately

$$X \rightarrow \text{window partition} \rightarrow \text{local attention} \rightarrow \text{window shift} \rightarrow \text{new local attention.} \quad (4.16)$$

Thus global information can propagate through repeated local interactions.

4.7 Hierarchical Representation

Swin also reduces spatial resolution between stages.

A patch-merging operation combines neighboring tokens.

For four neighboring representations,

$$x_1, x_2, x_3, x_4 \in \mathbb{R}^D, \quad (4.17)$$

concatenate them:

$$x_{\text{cat}} = [x_1; x_2; x_3; x_4] \in \mathbb{R}^{4D}. \quad (4.18)$$

Then project:

$$x' = x_{\text{cat}} W, \quad (4.19)$$

where

$$W \in \mathbb{R}^{4D \times 2D}. \quad (4.20)$$

Spatial resolution decreases while feature dimensionality increases.

4.8 Why the Hierarchy Matters

The resulting representation progresses from

$$\text{fine spatial resolution} \rightarrow \text{coarser spatial resolution} \rightarrow \text{higher-level representation.} \quad (4.21)$$

This resembles the hierarchical feature structure commonly found in convolutional networks.

4.9 Applications

Swin Transformer is particularly useful for:

- image classification,
- object detection,
- semantic segmentation,
- dense prediction,
- high-resolution computer vision.

Repository:

[Swin Transformer implementation](#)

Chapter 5

Pyramid Vision Transformer

5.1 Motivation

Many computer-vision tasks require hierarchical feature maps.

A classification model can operate primarily on a final global representation, but object detection and segmentation require features at multiple spatial scales.

Pyramid Vision Transformer (PVT) introduces a pyramid structure into Transformer-based vision models.

Paper:

Pyramid Vision Transformer: A Versatile Backbone for Dense Prediction without Convolutions

5.2 Hierarchical Input

The input image is transformed into patch embeddings.

Rather than maintaining one constant resolution,

$$H_1 \rightarrow H_2 \rightarrow H_3 \rightarrow H_4 \quad (5.1)$$

where

$$\text{resolution}(H_1) > \text{resolution}(H_2) > \text{resolution}(H_3) > \text{resolution}(H_4). \quad (5.2)$$

5.3 Spatial-Reduction Attention

PVT introduces spatial reduction for keys and values.

Queries are computed normally:

$$Q = HW^Q. \quad (5.3)$$

Instead of using all tokens to construct keys and values, a spatial reduction operator is applied:

$$H_R = \text{SR}(H). \quad (5.4)$$

Then

$$K = H_R W^K, \quad (5.5)$$

$$V = H_R W^V. \quad (5.6)$$

Attention becomes

$$Z = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V. \quad (5.7)$$

Since

$$|K| < |H|, \quad (5.8)$$

the attention computation is reduced.

5.4 Applications

PVT is designed as a general visual backbone, particularly for

- object detection,
- semantic segmentation,
- dense prediction,
- image classification.

Repository:

[PVT implementation](#)

Chapter 6

Deformable Attention Transformer

6.1 Motivation

Global attention considers a very large number of image locations.

However, an object or visual feature may only require information from a small set of relevant positions.

Deformable attention therefore attempts to learn which spatial locations should be sampled.

The concept is particularly important in detection-oriented Transformer architectures.

6.2 Reference Position

For a query at position p , define a reference point

$$p. \tag{6.1}$$

Instead of attending to every spatial location, the model predicts a set of offsets:

$$\Delta p_k = f_k(q). \tag{6.2}$$

The corresponding sampling locations are

$$p_k = p + \Delta p_k. \tag{6.3}$$

6.3 Sparse Sampling

Let the sampled values be

$$x(p_k). \tag{6.4}$$

Attention weights are

$$a_k = \text{softmax}_k(s_k). \tag{6.5}$$

The output becomes

$$y(p) = \sum_{k=1}^K a_k W_k x(p_k). \quad (6.6)$$

Only K selected positions contribute rather than all spatial locations.

Chapter 7

DETR

7.1 Motivation

Object detection traditionally uses several components:

$$\text{region proposals} + \text{classification} + \text{bounding-box regression} + \text{post-processing}. \quad (7.1)$$

DETR reframes object detection as a direct set-prediction problem.

Paper:

End-to-End Object Detection with Transformers

7.2 Image Encoder

An image

$$I \in \mathbb{R}^{H \times W \times C} \quad (7.2)$$

is processed by a backbone to obtain feature maps:

$$F \in \mathbb{R}^{H' \times W' \times D}. \quad (7.3)$$

Flatten spatial positions:

$$F \rightarrow X \in \mathbb{R}^{N \times D}. \quad (7.4)$$

Positional encodings are added:

$$X' = X + P. \quad (7.5)$$

7.3 Object Queries

DETR introduces a fixed set of learned object queries:

$$Q_{\text{obj}} = [q_1, \dots, q_M]. \quad (7.6)$$

These queries are not image patches.

They represent slots through which the decoder can predict objects.

7.4 Decoder Cross-Attention

For the object queries,

$$Q = Q_{\text{obj}} W^Q. \quad (7.7)$$

Image features provide keys and values:

$$K = X' W^K, \quad (7.8)$$

$$V = X' W^V. \quad (7.9)$$

Cross-attention is

$$A = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right), \quad (7.10)$$

and

$$Z = AV. \quad (7.11)$$

Each object query therefore extracts information from the image.

7.5 Prediction Heads

For each object query q_i , the decoder produces a representation

$$z_i. \quad (7.12)$$

A classification head predicts

$$p_i = \text{softmax}(W_c z_i + b_c). \quad (7.13)$$

A bounding-box head predicts

$$b_i = \sigma(W_b z_i + b_b), \quad (7.14)$$

where the predicted box can be parameterized as

$$b_i = (c_x, c_y, w, h). \quad (7.15)$$

7.6 Set Prediction

The key idea is that the predictions form an unordered set:

$$\hat{Y} = \{\hat{y}_1, \dots, \hat{y}_M\}. \quad (7.16)$$

Ground-truth objects form

$$Y = \{y_1, \dots, y_N\}. \quad (7.17)$$

A bipartite matching procedure determines the correspondence between predictions and ground-truth objects.

The training objective can therefore be expressed as

$$\mathcal{L} = \mathcal{L}_{\text{class}} + \lambda \mathcal{L}_{\text{box}} + \lambda' \mathcal{L}_{\text{GIoU}}. \quad (7.18)$$

7.7 Architectural Significance

DETR removes the need for a conventional region-proposal stage.

The conceptual pipeline is

$$\boxed{\text{image} \rightarrow \text{features} \rightarrow \text{object queries} \rightarrow \text{object set}}. \quad (7.19)$$

Repository:

[DETR implementation](#)

Chapter 8

MaskFormer

8.1 Motivation

Semantic segmentation predicts a class for every pixel.

Let

$$I \in \mathbb{R}^{H \times W \times C}. \quad (8.1)$$

The desired output is

$$Y \in \{1, \dots, C\}^{H \times W}. \quad (8.2)$$

MaskFormer reframes segmentation using mask predictions generated from Transformer queries.

Paper:

Per-Pixel Classification is Not All You Need for Semantic Segmentation

8.2 Pixel Features

An image encoder produces

$$F \in \mathbb{R}^{H' \times W' \times D}. \quad (8.3)$$

Flattening gives

$$F \rightarrow X \in \mathbb{R}^{N \times D}. \quad (8.4)$$

8.3 Mask Queries

Learned queries are

$$Q = [q_1, \dots, q_M]. \quad (8.5)$$

The Transformer decoder maps these queries into mask-level representations:

$$Z = \text{Decoder}(Q, X). \quad (8.6)$$

8.4 Mask Prediction

Let

$$z_i \tag{8.7}$$

be the representation of query i .

A mask embedding is generated:

$$m_i = W_m z_i. \tag{8.8}$$

Let pixel features be

$$f_p. \tag{8.9}$$

A mask logit can be computed through

$$s_{i,p} = m_i^T f_p. \tag{8.10}$$

The mask probability is

$$\hat{M}_{i,p} = \sigma(s_{i,p}). \tag{8.11}$$

Thus each query predicts a spatial mask.

Chapter 9

Multimodal Transformers

Vision Transformers can be extended beyond pure images.

Suppose an image produces

$$X_I \tag{9.1}$$

and text produces

$$X_T. \tag{9.2}$$

A multimodal Transformer must establish interactions between these representations.

There are several architectural possibilities:

1. concatenate the modalities;
2. perform cross-attention;
3. maintain separate encoders;
4. use a shared latent representation.

9.1 Cross-Attention

Suppose visual representations are

$$V \in \mathbb{R}^{n_v \times d}, \tag{9.3}$$

and textual representations are

$$T \in \mathbb{R}^{n_t \times d}. \tag{9.4}$$

Text queries can attend to image features:

$$Q_T = TW^Q, \tag{9.5}$$

$$K_V = VW^K, \tag{9.6}$$

$$V_V = VW^V. \quad (9.7)$$

Then

$$A_{T \leftarrow V} = \text{softmax} \left(\frac{Q_T K_V^T}{\sqrt{d_k}} \right), \quad (9.8)$$

and

$$Z_T = A_{T \leftarrow V} V_V. \quad (9.9)$$

The textual representation has therefore incorporated visual information.

Chapter 10

CLIP

10.1 Motivation

CLIP learns a shared representation for images and natural language.

Paper:

Learning Transferable Visual Models From Natural Language Supervision

The key architectural idea is not to force image and text tokens into the same Transformer at every layer.

Instead, separate encoders produce representations in a shared embedding space.

10.2 Image Encoder

The image encoder produces

$$z_I = f_I(I). \quad (10.1)$$

The representation is projected into the shared embedding space:

$$e_I = W_I z_I. \quad (10.2)$$

10.3 Text Encoder

A text encoder produces

$$z_T = f_T(T). \quad (10.3)$$

Projection gives

$$e_T = W_T z_T. \quad (10.4)$$

The representations are normalized:

$$\bar{e}_I = \frac{e_I}{\|e_I\|}, \quad (10.5)$$

$$\bar{e}_T = \frac{e_T}{\|e_T\|}. \quad (10.6)$$

10.4 Similarity

The image-text similarity is cosine similarity:

$$s(I, T) = \bar{e}_I^T \bar{e}_T. \quad (10.7)$$

A temperature parameter τ scales the logits:

$$\ell(I, T) = \frac{\bar{e}_I^T \bar{e}_T}{\tau}. \quad (10.8)$$

10.5 Contrastive Learning

For a batch of N image-text pairs, define

$$S_{ij} = \frac{\bar{e}_{I_i}^T \bar{e}_{T_j}}{\tau}. \quad (10.9)$$

The image-to-text loss is

$$\mathcal{L}_{I \rightarrow T} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(S_{ii})}{\sum_{j=1}^N \exp(S_{ij})}. \quad (10.10)$$

Similarly, the text-to-image loss is

$$\mathcal{L}_{T \rightarrow I} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(S_{ii})}{\sum_{j=1}^N \exp(S_{ji})}. \quad (10.11)$$

The total loss is

$$\mathcal{L} = \frac{1}{2} (\mathcal{L}_{I \rightarrow T} + \mathcal{L}_{T \rightarrow I}). \quad (10.12)$$

10.6 Zero-Shot Classification

Suppose the candidate classes are

$$c_1, \dots, c_K. \quad (10.13)$$

Construct text prompts

$$T_k = \text{“a photo of a } c_k\text{”}. \quad (10.14)$$

Their embeddings are

$$e_{T_k}. \quad (10.15)$$

Given an image embedding e_I , compute

$$s_k = \frac{e_I^T e_{T_k}}{\|e_I\| \|e_{T_k}\|}. \quad (10.16)$$

The predicted class is

$$\hat{k} = \arg \max_k s_k. \quad (10.17)$$

Thus the classifier does not necessarily require a separately trained classification head for every new task.

Repository:

[OpenAI CLIP implementation](#)

Chapter 11

BLIP and Vision-Language Transformer Architectures

11.1 Motivation

Vision-language systems require more than image classification.

Typical tasks include

- image captioning,
- visual question answering,
- image-text retrieval,
- image-grounded generation.

BLIP introduces an architecture designed to support multiple vision-language objectives.

Paper:

BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation

11.2 Vision Encoder

An image is converted into visual representations:

$$I \rightarrow V = [v_1, \dots, v_m]. \quad (11.1)$$

11.3 Text Representation

Text is represented as

$$T = [t_1, \dots, t_n]. \quad (11.2)$$

A language Transformer processes the textual sequence.

11.4 Image-Text Interaction

Cross-attention allows textual queries to retrieve visual information.

$$Q_T = TW^Q, \quad (11.3)$$

$$K_V = VW^K, \quad (11.4)$$

$$V_V = VW^V. \quad (11.5)$$

Then

$$Z_T = \text{softmax} \left(\frac{Q_T K_V^T}{\sqrt{d_k}} \right) V_V. \quad (11.6)$$

The resulting representation contains both textual and visual information.

Repository:

[BLIP implementation](#)

Chapter 12

BEiT

12.1 Motivation

Standard ViT pretraining can use image classification labels.

BEiT instead investigates masked image modeling, drawing an analogy with masked language modeling.

Paper:

BEiT: BERT Pre-Training of Image Transformers

12.2 Image Tokenization

An image is divided into patches:

$$I \rightarrow (x_1, \dots, x_N). \quad (12.1)$$

A discrete visual tokenizer maps patches to visual tokens:

$$z_i = g(x_i). \quad (12.2)$$

Thus

$$z_i \in \{1, \dots, K\}. \quad (12.3)$$

12.3 Masking

A subset

$$\mathcal{M} \subset \{1, \dots, N\} \quad (12.4)$$

is masked.

The Transformer receives corrupted image tokens.

Conceptually,

$$\tilde{x}_i = \begin{cases} \text{MASK}, & i \in \mathcal{M}, \\ x_i, & i \notin \mathcal{M}. \end{cases} \quad (12.5)$$

12.4 Prediction

The Transformer produces

$$H = f_\theta(\tilde{X}). \quad (12.6)$$

For masked position i ,

$$p_\theta(z_i \mid \tilde{X}) = \text{softmax}(Wh_i + b). \quad (12.7)$$

The training loss is

$$\mathcal{L} = - \sum_{i \in \mathcal{M}} \log p_\theta(z_i \mid \tilde{X}). \quad (12.8)$$

The model therefore learns to infer missing visual tokens from their surrounding context.

Repository:

[BEiT implementation](#)

Chapter 13

MAE: Masked Autoencoders

13.1 Motivation

MAE applies masked reconstruction to images using an asymmetric encoder-decoder architecture.

Paper:

Masked Autoencoders Are Scalable Vision Learners

13.2 Patch Sequence

The image becomes

$$X = [x_1, \dots, x_N]. \quad (13.1)$$

Choose a visible subset

$$\mathcal{V} \quad (13.2)$$

and a masked subset

$$\mathcal{M}. \quad (13.3)$$

Thus

$$\mathcal{V} \cup \mathcal{M} = \{1, \dots, N\}. \quad (13.4)$$

13.3 Encoder

Only visible patches are sent through the encoder:

$$Z = f_{\text{enc}}(X_{\mathcal{V}}). \quad (13.5)$$

Therefore the encoder does not process the full patch sequence.

13.4 Decoder

Mask tokens are inserted for the missing patches.

The decoder input can be represented as

$$\tilde{Z} = [Z; M_{\mathcal{M}}] + E_{\text{pos}}. \quad (13.6)$$

The decoder produces

$$Y = f_{\text{dec}}(\tilde{Z}). \quad (13.7)$$

13.5 Pixel Reconstruction

For a masked patch x_i , the decoder predicts

$$\hat{x}_i. \quad (13.8)$$

A reconstruction loss can be written as

$$\mathcal{L}_{\text{MAE}} = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \|x_i - \hat{x}_i\|_2^2. \quad (13.9)$$

Only masked patches contribute to the reconstruction objective.

13.6 Why the Asymmetry Matters

The encoder sees only visible patches:

$$X_{\mathcal{V}} \rightarrow Z. \quad (13.10)$$

The decoder receives the latent representation together with mask tokens:

$$(Z, M_{\mathcal{M}}) \rightarrow \hat{X}_{\mathcal{M}}. \quad (13.11)$$

Therefore the expensive encoder operates on substantially fewer tokens.

Repository:

[MAE implementation](#)

Chapter 14

DINO and Self-Supervised Vision Transformers

14.1 Motivation

Transformers can also learn visual representations without explicit human-provided labels.

DINO uses self-distillation between different views of the same image.

Paper:

Emerging Properties in Self-Supervised Vision Transformers

14.2 Multiple Views

Given an image x , generate different augmented views:

$$x_1 = t_1(x), \tag{14.1}$$

$$x_2 = t_2(x). \tag{14.2}$$

The student receives one view while the teacher receives another.

14.3 Student and Teacher

The student produces

$$p_S = f_S(x_1). \tag{14.3}$$

The teacher produces

$$p_T = f_T(x_2). \tag{14.4}$$

The student is trained to match the teacher distribution.

A cross-entropy objective is

$$\mathcal{L} = - \sum_i p_T(i) \log p_S(i). \quad (14.5)$$

The teacher parameters are updated using a moving-average style update rather than ordinary backpropagation.

Conceptually,

$$\theta_T \leftarrow \lambda \theta_T + (1 - \lambda) \theta_S. \quad (14.6)$$

14.4 Architectural Significance

The important modification is not a new attention equation.

Instead, the Transformer becomes part of a self-supervised teacher-student system.

The progression is

$$\text{image views} \rightarrow \text{student/teacher Transformers} \rightarrow \text{representation agreement}. \quad (14.7)$$

Repository:

[DINO implementation](#)

Chapter 15

Overall Architectural Progression

The major vision-oriented Transformer architectures discussed in this part modify different stages of the original pipeline.

15.1 ViT

ViT changes

$$\text{image} \rightarrow \text{patch tokens.} \quad (15.1)$$

The Transformer itself remains close to the original encoder.

15.2 Swin

Swin changes the attention pattern:

$$\text{global attention} \rightarrow \text{window attention} \rightarrow \text{shifted-window attention.} \quad (15.2)$$

It also introduces hierarchical spatial resolution.

15.3 PVT

PVT introduces hierarchical features together with reduced key-value resolution:

$$K, V \leftarrow \text{SR}(X). \quad (15.3)$$

15.4 DETR

DETR changes object detection from a collection of proposal-based operations into

$$\text{image} \rightarrow \text{object queries} \rightarrow \text{set of objects.} \quad (15.4)$$

15.5 CLIP

CLIP separates image and text encoding:

$$I \rightarrow e_I, \quad T \rightarrow e_T, \quad (15.5)$$

and learns the shared embedding space through contrastive learning.

15.6 MAE

MAE changes the pretraining objective:

$$\text{visible patches} \rightarrow \text{latent representation} \rightarrow \text{reconstruction of masked patches}. \quad (15.6)$$

15.7 Multimodal Transformers

Multimodal models introduce cross-modal information flow:

$$X_A \rightarrow Q_A, \quad X_B \rightarrow K_B, V_B, \quad (15.7)$$

followed by

$$\text{Attention}(Q_A, K_B, V_B). \quad (15.8)$$

Thus the Transformer framework can be modified at several different levels:

input representation,
 attention pattern,
 sequence hierarchy,
 query structure,
 memory mechanism,
 training objective,
 modality interaction.

(15.9)